Exp. No: 4.A

SUPPORT VECTOR MACHINES (SVM)

Code:

```
# 1. Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
  accuracy_score,
  precision_score,
  recall_score,
  f1_score,
  confusion_matrix,
  classification_report,
  roc_auc_score,
  roc_curve
)

# 2. Load Dataset
data = load_breast_cancer()

X = pd.DataFrame(
  data.data,
  columns=data.feature_names
)
```

```
y = pd.Series(
  data.target,
  name="target"
)

# 0 = Malignant
# 1 = Benign

# 3. Train/Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.20,
  random_state=42,
  stratify=y
)

# 4. Standardize Features
scaler = StandardScaler()

X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# 5. Define SVM Model with RBF Kernel
svm = SVC(
  kernel="rbf",
  probability=True,
  random_state=42
)
# 6. Define Hyperparameter Grid
param_grid = {
  "C": [0.1, 1, 10, 100],
  "gamma": ["scale", 0.01, 0.001, 0.0001]
}
```

```
# 7. Hyperparameter Tuning using GridSearchCV
grid = GridSearchCV(
  estimator=svm,
  param_grid=param_grid,
  scoring="f1",
  cv=5,
  n_jobs=-1,
  verbose=0
)

# Train Grid Search
grid.fit(X_train_sc, y_train)

# Display Best Parameters
print("Best Parameters from Grid Search:")
print(grid.best_params_)

# Get Best Model
best_svm = grid.best_estimator_

# 8. Train Final SVM Model
best_svm.fit(X_train_sc, y_train)

# Predictions
y_pred = best_svm.predict(X_test_sc)

# Probability predictions for ROC-AUC
y_prob = best_svm.predict_proba(X_test_sc)[:, 1]

# 9. Calculate Evaluation Metrics

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(
  y_test,
  y_pred,
  zero_division=0
)
```

```
recall = recall_score(
  y_test,
  y_pred,
  zero_division=0
)

f1 = f1_score(
  y_test,
  y_pred,
  zero_division=0
)

auc = roc_auc_score(
  y_test,
  y_prob
)

cm = confusion_matrix(
  y_test,
  y_pred
)

# 10. Display Results

print("\n========================================")
print("   SVM (RBF) — TEST METRICS")
print("========================================")

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall  : {recall:.4f}")
print(f"F1-Score : {f1:.4f}")
print(f"ROC-AUC : {auc:.4f}")

print("\nConfusion Matrix:")
print(cm)
```

```
print("\nClassification Report:")
print(
  classification_report(
    y_test,
    y_pred,
    target_names=["Malignant", "Benign"],
    zero_division=0
  )
)

# 11. Plot ROC Curve

fpr, tpr, thresholds = roc_curve(
  y_test,
  y_prob
)

plt.figure(figsize=(7, 5))

plt.plot(
  fpr,
  tpr,
  label=f"SVM (AUC = {auc:.3f})"
)

plt.plot(
  [0, 1],
  [0, 1],
  linestyle="--",
  color="gray",
  label="Random Classifier"
)
```

```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve — SVM (RBF)")

plt.legend()
plt.grid(True)

plt.show()
```

Output:

```
Best Parameters from Grid Search:
{'C': 10, 'gamma': 0.01}

========================================
      SVM (RBF) — TEST METRICS
========================================
Accuracy  : 0.9825
Precision : 0.9861
Recall    : 0.9861
F1-Score  : 0.9861
ROC-AUC   : 0.9977

Confusion Matrix:
[[41  1]
 [ 1 71]]

Classification Report:
              precision    recall  f1-score   support

   Malignant       0.98      0.98      0.98        42
      Benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

